

CHAIN POKER GENESIS BY LAEV (peroqtdigo)

Table Wallet Engine

Official Technical Specification v1.0

1. Purpose

The **Table Wallet Engine** is the protocol component responsible for creating, managing, renewing, and destroying the temporary shared wallet used exclusively for the monetary settlement of each poker hand.

The table wallet is independent from the personal wallets of the player nodes and exists only while the composition of participants at the table remains valid.

Its purpose is to temporarily hold the satoshis wagered during a hand and automatically distribute them once settlement has been completed.

2. Fundamental Principle

Within Chain Poker Genesis there are two types of wallets:

A. Player Node Wallet

This wallet belongs exclusively to the player.

It allows the player to:

- maintain a personal balance;
- open payment channels;
- purchase virtual chips represented in satoshis;
- receive winnings;
- pay rake.

It never belongs to the table.

B. Table Wallet

The table wallet belongs exclusively to the poker table.

It does not maintain a permanent balance.

Its lifecycle begins before a hand starts and ends immediately after the settlement has been fully completed.

At the end of each hand, the expected wallet balance is zero.

3. Temporary Nature

Each table wallet represents an exact snapshot of the players seated at the table.

If a player:

- joins;
- leaves;
- loses connection;
- changes seats;

the cryptographic composition changes.

As a consequence, the next hand will use a new table wallet.

4. Recommended Technology

The preferred implementation consists of using a key aggregation scheme based on aggregated signatures (for example, MuSig2 over Taproot) or equivalent mechanisms supported by the selected Layer 2 infrastructure, instead of a traditional N-of-M multisignature.

This approach provides:

- a single public key representing the table;
- a single aggregated signature to authorize settlement;

- smaller transaction sizes;
- improved privacy;
- efficient renewal whenever the player composition changes.

If the selected Layer 2 infrastructure does not support this scheme, a traditional multisignature implementation adapted to the capabilities of that network may be used.

5. Creation

Before each hand begins, the engine generates:

- Table Wallet ID;
- aggregated public key of the table;
- cryptographic identifier;
- participant list;
- configuration version;
- initial state.

The wallet is then ready to receive wagers.

6. Receiving Funds

During the hand, every wager placed by a player is transferred in real time from the player's payment channel or available balance to the temporary table wallet.

This includes, among other betting actions:

- ante;
- small blind;
- big blind;
- call;

- raise;
- re-raise;
- all-in.

At all times, the wallet reflects the current size of the pot.

7. Temporary Custody

The table wallet acts solely as the temporary custodian of the pot.

It does not perform partial payouts during the hand.

All movements remain recorded within the Off-Chain Private Ledger.

8. Settlement

When the Poker Rules Engine determines the final outcome:

- single winner;
- multiple winners;
- side pots;
- split pot;
- return of wagers when applicable;

the Monetary Settlement Engine calculates the final distribution.

The table wallet then performs the corresponding payments to:

- winning wallets;
- rake wallet;
- authorized refunds.

9. Destruction

Once settlement has been completed:

- the balance must be equal to zero;
- the status changes to **CLOSED**;
- the wallet is no longer used;
- only its historical record remains.

The next hand will use a new table wallet if the player composition has changed.

10. Renewal

The engine automatically creates a new table wallet whenever any of the following events occurs:

- a player joins;
- voluntary departure;
- player removal;
- permanent disconnection;
- modification of the authorized cryptographic composition.

If the player composition remains identical between hands, the protocol may reuse the same cryptographic configuration or generate a new one according to the defined security policy.

11. Integration

The Table Wallet Engine interacts directly with:

- Table Join Engine;
- Communications Engine;
- Off-Chain Private Ledger Engine;
- Monetary Settlement Engine;

- Poker Rules Engine;
- Consensus Engine.

12. Principles

The engine is designed around the following principles:

- temporary custody;
- absence of permanent balances;
- independence from personal wallets;
- dynamic participant composition;
- privacy;
- cryptographic traceability;
- deterministic settlement;
- compatibility with Bitcoin Layer 2 infrastructures.

Each hand represents a complete financial cycle: the table wallet receives the wagered satoshis, safeguards the pot while the hand is active, distributes the funds according to the rules of the game, and concludes with a zero balance, ensuring that no table retains residual funds between hands.

Traducir al ingles

CHAIN POKER GENESIS BY LAEV (peroqtdigo)

Motor de Creación de Billetera de Mesa (Table Wallet Engine)

Especificación Técnica Oficial v1.0

1. Propósito

El Motor de Creación de Billetera de Mesa es el componente del protocolo responsable de crear, administrar, renovar y destruir la billetera grupal temporal utilizada exclusivamente para la liquidación monetaria de cada mano de poker.

La billetera de mesa es independiente de las billeteras personales de los nodos jugadores y únicamente existe mientras la composición de participantes de la mesa permanezca válida.

Su objetivo es custodiar temporalmente los satoshis apostados durante una mano y distribuirlos automáticamente al finalizar la liquidación.

2. Principio Fundamental

Dentro de Chain Poker Genesis existen dos tipos de billeteras:

A. Billetera del Nodo Jugador

Pertenece únicamente al jugador.

Permite:

- mantener saldo personal;
- abrir canales de pago;
- comprar fichas virtuales representadas en satoshis;
- recibir premios;
- pagar rake.

Nunca pertenece a la mesa.

B. Billetera de Mesa

La billetera de mesa pertenece exclusivamente a la mesa de juego.

No mantiene saldo permanente.

Su ciclo de vida inicia antes de una mano y finaliza inmediatamente después de la liquidación completa.

Al terminar una mano, el saldo esperado de la billetera es cero.

3. Naturaleza Temporal

Cada billetera de mesa representa una fotografía exacta de los jugadores sentados.

Si un jugador:

- entra,
- sale,
- pierde conexión,
- cambia de asiento,

la composición criptográfica cambia.

Como consecuencia, la siguiente mano utilizará una nueva billetera de mesa.

4. Tecnología Recomendada

La implementación preferente consiste en utilizar una agregación de claves basada en firmas agregadas (por ejemplo, MuSig2 sobre Taproot) o mecanismos equivalentes soportados por la infraestructura L2 elegida, en lugar de una multifirma clásica N-de-M.

Este enfoque proporciona:

- una única clave pública visible para la mesa;
- una única firma agregada para autorizar la liquidación;
- menor tamaño de transacción;
- mejor privacidad;
- renovación eficiente cuando cambia la composición de jugadores.

En caso de que la infraestructura L2 seleccionada no soporte este esquema, podrá utilizarse una multifirma tradicional adaptada a las capacidades de dicha red.

5. Creación

Antes del inicio de cada mano el motor genera:

- Table Wallet ID;
- clave pública agregada de la mesa;
- identificador criptográfico;
- lista de participantes;
- versión de configuración;
- estado inicial.

La billetera queda lista para recibir apuestas.

6. Recepción de Fondos

Durante la mano, cada apuesta realizada por un jugador es enviada en tiempo real desde su canal o saldo disponible hacia la billetera temporal de mesa.

Entre otros conceptos:

- ante;
- small blind;
- big blind;
- call;
- raise;
- re-raise;
- all-in.

La billetera refleja en todo momento el pozo vigente.

7. Custodia Temporal

La billetera de mesa actúa únicamente como custodio temporal del pozo.

No realiza pagos parciales durante la mano.

Todos los movimientos permanecen registrados dentro del Ledger Privado Off-Chain.

8. Liquidación

Cuando el Motor de Reglas de Poker determina el resultado definitivo:

- ganador único;
- múltiples ganadores;
- side pots;
- empate;
- devolución de apuestas cuando corresponda,

el Motor de Liquidación Monetaria calcula la distribución final.

Posteriormente la billetera de mesa realiza los pagos correspondientes:

- billeteras ganadoras;
- billetera de rake;
- devoluciones autorizadas.

9. Destrucción

Una vez finalizada la liquidación:

- el saldo debe ser igual a cero;
- el estado pasa a CLOSED;
- la billetera deja de utilizarse;
- únicamente permanece su registro histórico.

La siguiente mano utilizará una nueva billetera si la composición de jugadores cambió.

10. Renovación

El motor crea automáticamente una nueva billetera cuando ocurra cualquiera de los siguientes eventos:

- ingreso de un jugador;
- salida voluntaria;
- expulsión;
- desconexión definitiva;
- modificación de la composición criptográfica autorizada.

Si la composición permanece idéntica entre manos, el protocolo puede reutilizar la misma configuración criptográfica o generar una nueva, según la política de seguridad definida.

11. Integración

El Motor de Creación de Billetera de Mesa interactúa directamente con:

- Motor de Incorporación de Jugadores;
- Motor de Comunicaciones;
- Motor de Ledger Privado Off-Chain;
- Motor de Liquidación Monetaria;
- Motor de Reglas de Poker;
- Motor de Consenso.

12. Principios

El diseño del motor se fundamenta en:

- custodia temporal;
- ausencia de saldo permanente;
- independencia respecto a las billeteras personales;

- composición dinámica de participantes;
- privacidad;
- trazabilidad criptográfica;
- liquidación determinística;
- compatibilidad con infraestructuras Bitcoin de segunda capa.

Cada mano representa un ciclo financiero completo: la billetera de mesa recibe los satoshis apostados, conserva el pozo mientras la mano está activa, distribuye los fondos conforme a las reglas del juego y concluye con saldo cero, garantizando que ninguna mesa mantenga fondos residuales entre manos.